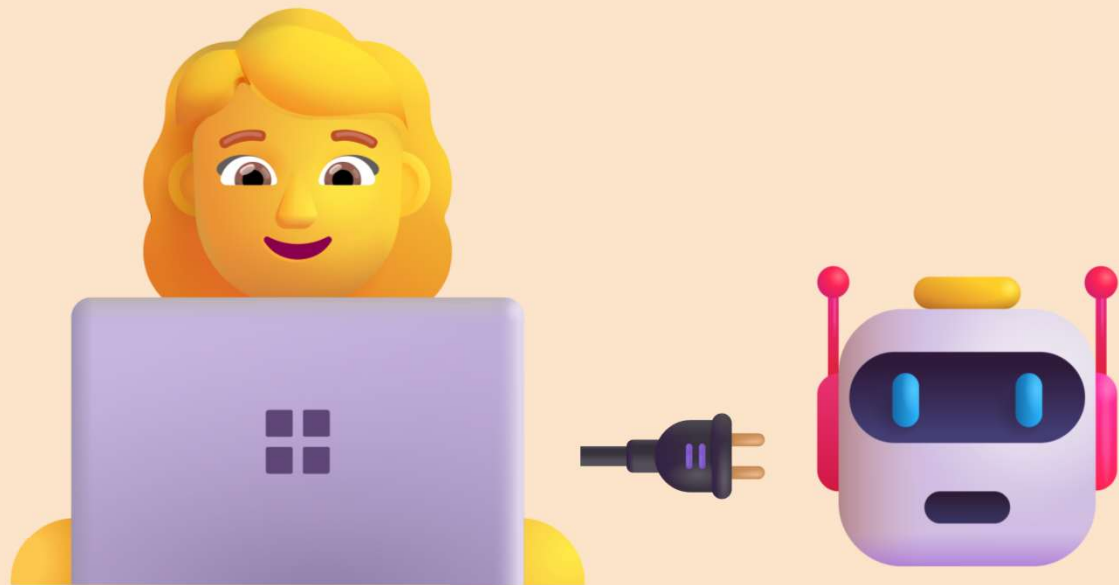


AI Insights

MCP – Model Context Protocol
...Ein Jahr später.

Oliver Welz (ScaDS.AI)



Worum geht's?

Model Context Protocol –
recap was das ist,
was sich daran getan hat
...und wie das umgesetzt
wurde.





Kurze Abfrage:

- 1 ... Wer **nutzt** MCP bereits &
- 2 ... Was stellst **du** damit so an?



Diskussion:

Auf **welche Grenzen** bist du bei der **Nutzung** von **MCP** gestoßen?

Hast du es gelöst und wenn ja, wie?

In einem Jahr hat sich viel
getan bzgl. **MCP** – Was genau?
*...dem wollen wir uns nun im
folgenden widmen:*



*...Aber von
vorne*

Definition
... *MCP*





Was ist das *Model Context Protocol?*

...**Definition** und erste
Erklärungen

MCP ist ein offenes Protokoll, das KI-Hosts (LLMs/Agenten, IDEs, Apps) standardisiert mit **Tools, Datenquellen und Prompt-Vorlagen** verbindet. ¹

Es ermöglicht **strukturierte, bidirektionale Aufrufe**: **Modelle** können **Kontext holen** (Resources/Prompts) und **Aktionen ausführen** (Tools) – **modell-agnostisch** und in Echtzeit. ^{1, 2}

Ziel: das **N×M-Integrationsproblem** auflösen – einmal als **MCP-Server** anbinden, von vielen Modellen/Clients **wiederverwendbar, reproduzierbar** und **auditierbar**.

Sicherheit & Governance eingebaut: Schemas, Scopes/Berechtigungen, Logs; optional **Human-in-the-Loop** – für **mehr Autonomie bei hoher Verlässlichkeit**.

Was es nicht ist: ein eigenes KI-Modell, Agenten-Framework oder proprietärer Plugin-Store – **sondern die „Standard-Schnittstelle“ (Protokoll)** zwischen Modell und *Welt*.

sources:

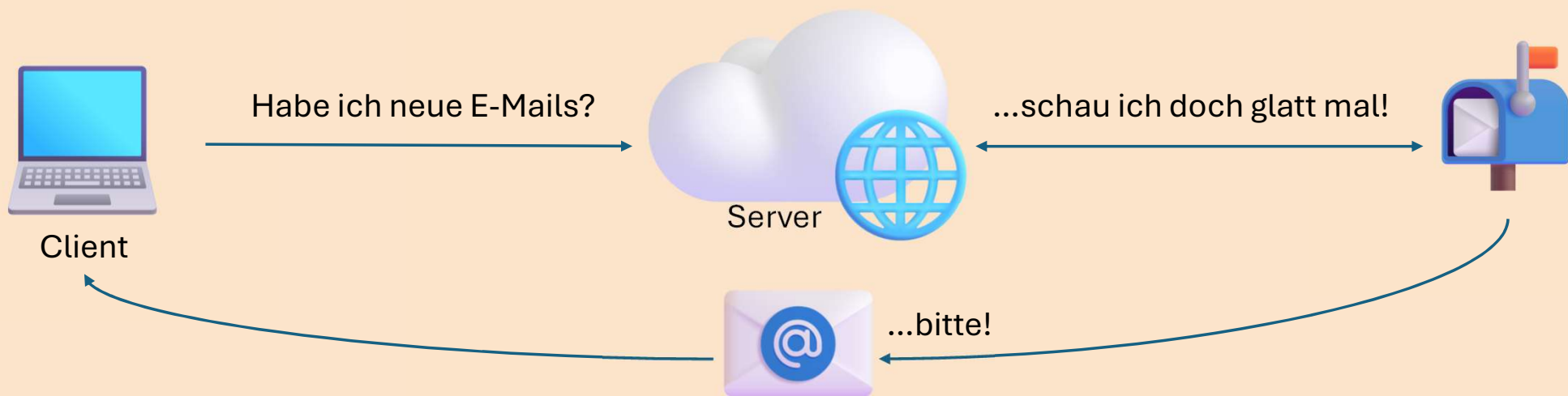
1. <https://modelcontextprotocol.io/docs/getting-started/intro>
2. <https://github.com/modelcontextprotocol/modelcontextprotocol>

„MCP also...”

...und **wie** geht das jetzt und *wie*
kann ich mir das vorstellen!?



Ihr kennt das alle schon *laaange* und nutzt es täglich- z.B.:
...wenn ihr eure **E-Mails** abrufen:



Client = die anfordernde Seite (UI / Agent / App) → stellt **strukturierte Anfragen**.

Server = die ausführende Seite (Tool / API / Service / LLM-Gateway); stellt alle Funktionen bereit
→ liefert **strukturierte Antworten**.

Protokoll: Beide Seiten sprechen ein **klar definiertes Format** (Schema für Request/Response: Felder, Typen, Fehlercodes).

Warum das *hilft* und funktioniert:

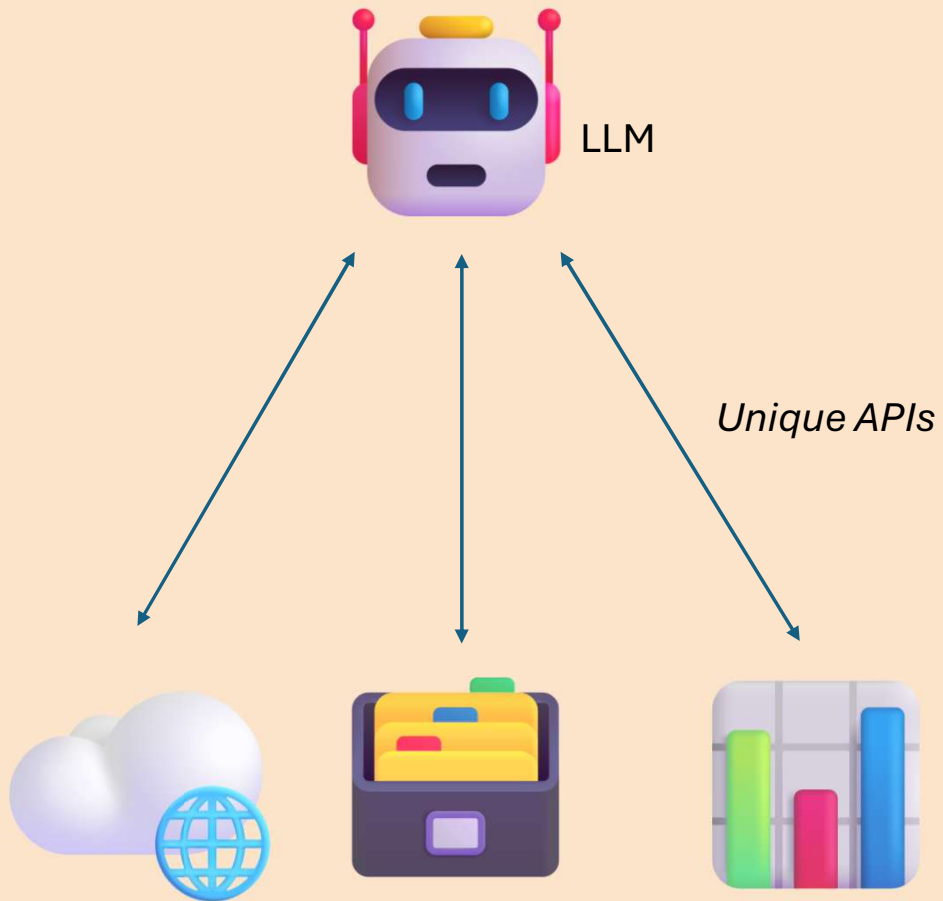
- ✓ **Eindeutig statt nicht deterministisch:** Parameter statt Freitext → weniger Missverständnisse.
- ✓ **Validierbar:** Falsche/fehlende Angaben werden früh entdeckt.
- ✓ **Austauschbar:** Tools können hinter dem gleichen Vertrag ersetzt/erweitert werden.
- ✓ **Beobachtbar & sicher:** IDs, Logs, Status; Policies begrenzen, was erlaubt ist.
- ✓ **Reproduzierbar:** Gleiche Anfrage → gleiche Wirkung (besser testen & debuggen).

„MCP...“

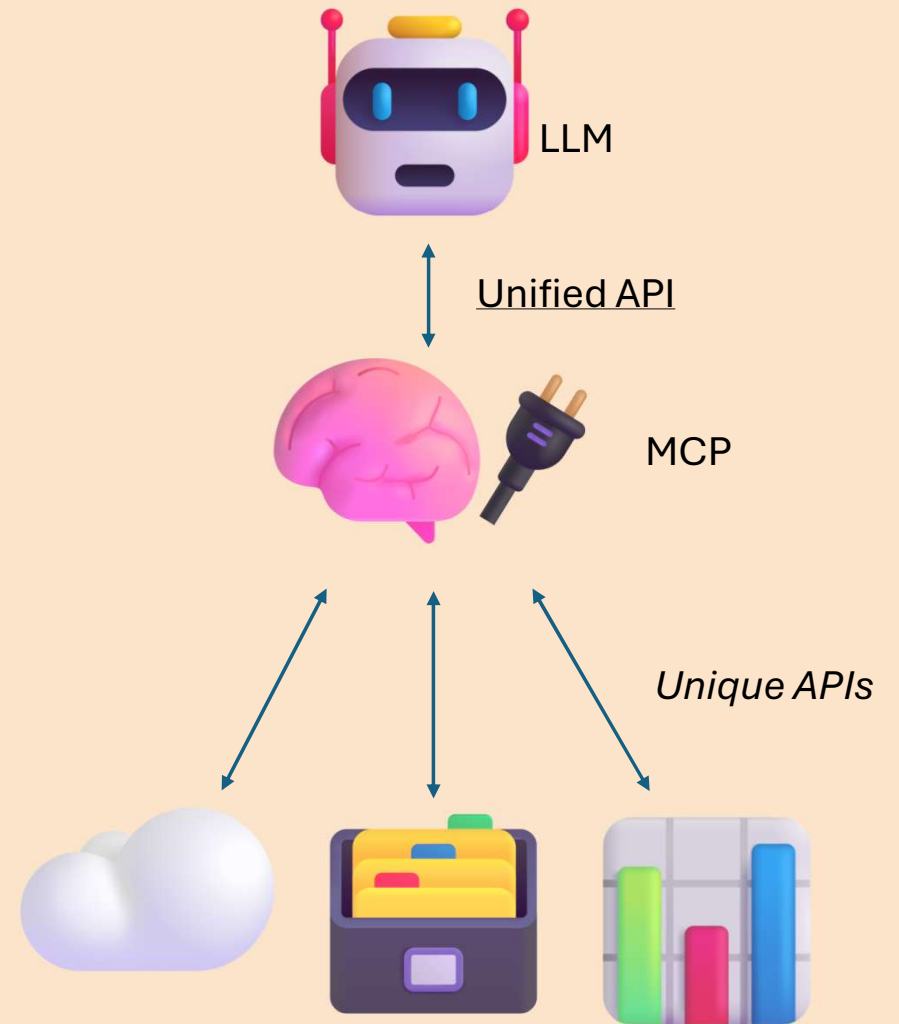
...schematisch dar- und
gegenübergestellt.



Ohne MCP

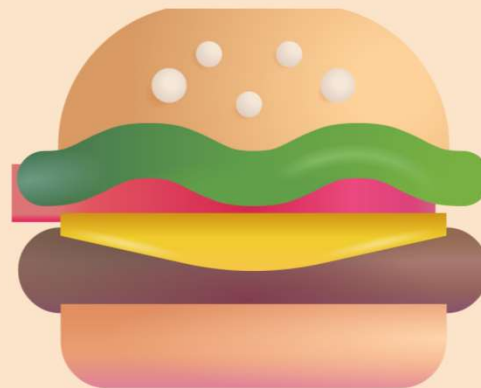


Mit MCP



**Und da das jetzt sicher
nicht für alle greifbar war,**

...nutzen wir ein Gleichnis, um
uns das nochmal genauer
anzusehen.



Willkommen im Burger-Schnellrestaurant:

Früher – der Schalter, manuelle Arbeit (sprich ohne MCP):

 Kunde erklärt →  Kassierer/in tippt →  Küche erhält Bestellung

✗ Missverständnisse, Rückfragen, Wartezeit.

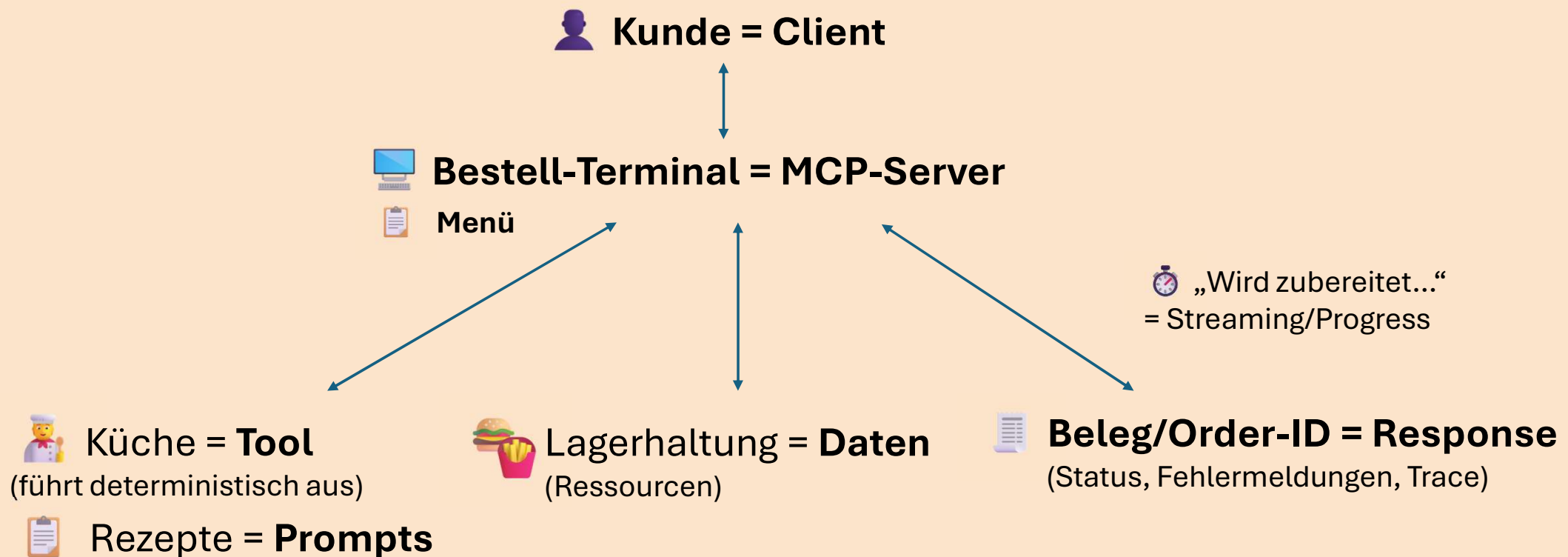
→ Jede/r Kassierer/in muss jede Bestellung „verstehen“ und korrekt weitergeben

(n Kundenbestellungen × m Mitarbeitende).

...kompliziert & potenziell fehleranfällig

Willkommen im Burger-Schnellrestaurant:

Heute – über Bestellterminal (sprich mit MCP → standardisiert):





Willkommen im Burger-Schnellrestaurant:

Takeaway:

Ein **einheitliches** (Bestell-)Protokoll macht Abläufe **skalierbar, überprüfbar und austauschbar**.

Nicht nur die Nutzung von **Tools**, sondern auch bspw. Datenzugriff und Bearbeitung werden **einheitlich und stabil**

„Okay...“

...stellen wir nun **gegenüber, was**
sich auf **technischer Ebene wie**
verändert hat:





**Stateful vs.
Stateless**

MCP 2025: *stateful* gedacht:

→ initialize-Handshake → Version & Capabilities
aushandeln (optional Session-ID)

Folge-Requests konnten auf bestehendem Verbindungszustand aufbauen, was praktisch bei lokalen / kleinen Setups ist aber problematischer bei verteilten Remote-Systemen

Warum Stateless? – besser horizontal skalieren

- **Stateful:** Folge-requests hängen am bisherigen Serverzustand & dadurch braucht man **Sticky Sessions** oder einen **gemeinsamen Session Store**
- **Stateless:** jeder Request enthält alles Nötige selbst & deshalb kann **jeder freie Server** jeden Request übernehmen neue Instanzen lassen sich leichter hinzufügen, ersetzen oder entfernen

→ **Autoscaling, Failover und Load Balancing werden einfacher**



**Stateful vs.
Stateless**

MCP 2026: jeder Request steht für sich

- **kein initialize-Handshake mehr**
 - **keine zwangsläufigen MCP-Session-ID mehr**
- Version, Client-Info & Capabilities kommen mit jedem Request
- jeder Request ist **für sich interpretierbar** kein vorheriger MCP-Sessionzustand nötig
- **jeder Worker kann jeden Request bearbeiten**

Stateless heißt nicht zustandslos

- Protokoll ist *stateless*; Anwendung darf weiterhin **State** haben
- jeder Request ist **eigenständig verständlich**
- laufende Vorgänge bleiben möglich
- Zustand über explizite IDs / Handles

State liegt in Anwendung / Backend, nicht im Protokoll

MCP ist stateless – die Anwendung muss es nicht sein
→ **Stateless Communication, expliziter Anwendungszustand**





Multi
Round-Trip
Request

MRTR – Rückfragen ohne laufende Session

- Server braucht noch Information oder Freigabe
 - Antwort: „**input required**“
 - Client holt die fehlende Information ein
 - Anfrage wird mit der Antwort erneut gesendet
- optionaler requestState trägt die Fortsetzungsinfo mit

Vorteil:

Server muss **keinen offenen Vorgang** im **Speicher behalten**



Streamable HTTP

Streamable HTTP

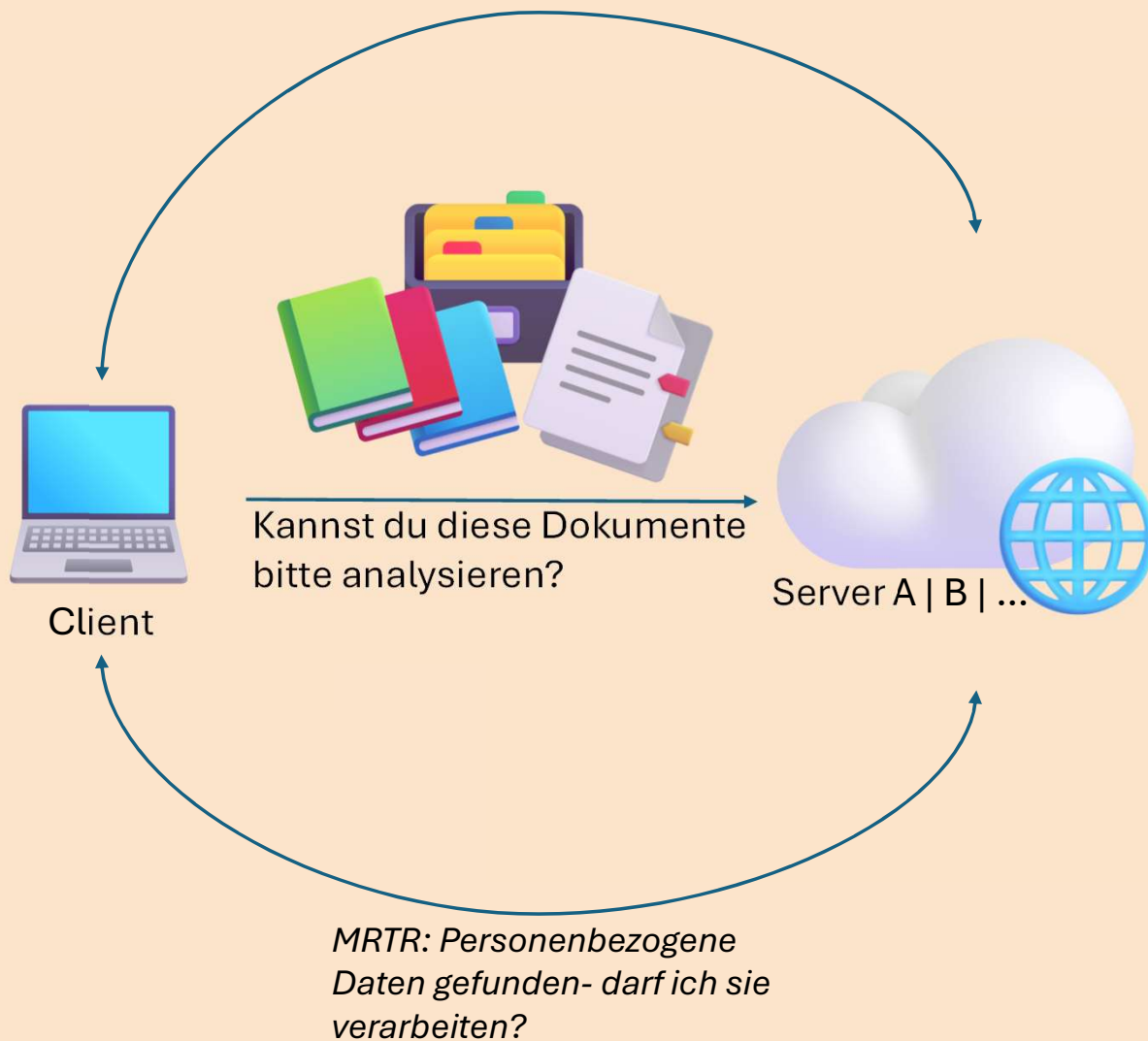
- *alter HTTP+SSE-Transport* ist *deprecated*
- jeder MCP-Aufruf ist ein **eigener HTTP-Request**
- keine dauerhaft offene MCP-Verbindung / Session mehr
 - Antwort kommt direkt oder wird **nur für diesen Request gestreamt** → endet der Request, endet auch der Stream
- nächster MCP-Aufruf ist wieder unabhängig
 - besser für **Statelessness, Load Balancing** und **horizontale Skalierung** bei gleichzeitiger Steigerung der **Interaktivität**

„Okay...“

...zeigen wir das in einem
Beispiel !



Streamable HTTP für den
Bearbeitungsfortschritt



- **Streaming begleitet einen Request.**
- **MRTR verbindet mehrere Requests.**
- **Statelessness** sorgt dafür, dass dafür **keine dauerhafte Server-Session nötig ist.**

Weitere Neuerungen³: mehr Production-Readiness & mehr etablierte Web-/Cloud-Standards

- **Routing:** MCP-Method / MCP-Name machen Requests für Gateways, Policies, Rate-Limits etc. sichtbar
- **Caching:** Tool- und Resource-Listen können gezielter gecacht werden
- **Tracing:** W3C Trace Context verbessert verteilte Nachvollziehbarkeit
- Einige **MCP-Sonderfunktionen** sind wieder **rausgenommen** (z.B. MCP eigenes Logging: jetzt ersetzt durch OpenTelemetry)



sources:

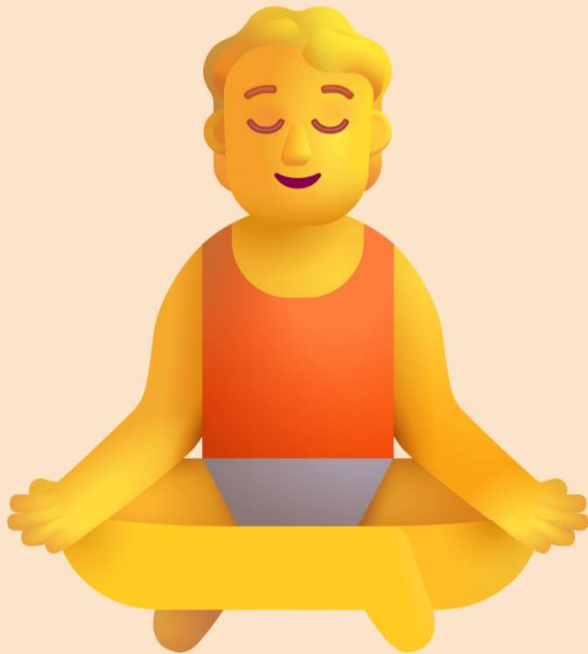
3. <https://modelcontextprotocol.io/specification/2026-07-28/changelog>

Skalierungsproblem das bleibt: Tool Explosion | Context

- MCP macht Tool-Integration einfacher & dadurch wachsen Tool-Kataloge
- Tool-Schemas verbrauchen Kontext
- ähnliche Tools erschweren Auswahl
- mehr Tokens, mehr mögliche Fehlselektion
- MCP löst Integration – nicht automatisch Tool Selection

→ **Wie** man das **lösen kann**, zeigen wir mit **Progressive Disclosure** im „**advanced Track**“ des **Workshops**.





...durchatmen

Gibt es *Fragen?*
Kommentare?

Also,

...fassen wir
zusammen

MCP ist 2026 deutlich produktionsnäher

- weniger impliziter Zustand
- besser skalierbar
- besser integrierbar in bestehende Web-Infrastruktur

aber Herausforderungen bleiben:

- Tool Selection
- Context Management
- Orchestrierung
- Sicherheit



Was MCP nicht löst

- wählt nicht **automatisch** das *richtige Tool*
- garantiert keine korrekten Agentenentscheidungen
- garantiert keine sicheren Tool-Inhalte
- verhindert **Prompt Injection** nicht
- ersetzt Auth / Authorization / Policies nicht
- ersetzt Human-in-the-Loop nicht

Standardisierung ≠ Vertrauen

Und Achtung,

...Vorsicht ist je nach
Integration und Zugriff des
LLMs geboten

Handlungsfähigkeit braucht Leitplanken

Rechte & Zugriff

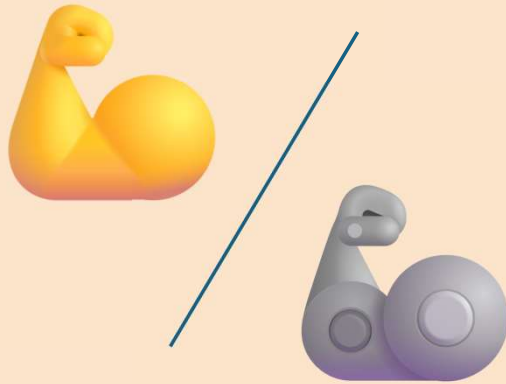
- Least Privilege statt Over-Permissioning
- sensible / mutierende Aktionen gezielt freigeben
- Human-in-the-Loop

Robustheit

- Schema- und Argumentvalidierung
- Limits / Quotas
- Idempotenz bei Wiederholungen
- Abbruch- und Retry-Strategien

Trust

- Tool-Inhalte und Ergebnisse als potenziell untrusted behandeln
- Prompt Injection mitdenken

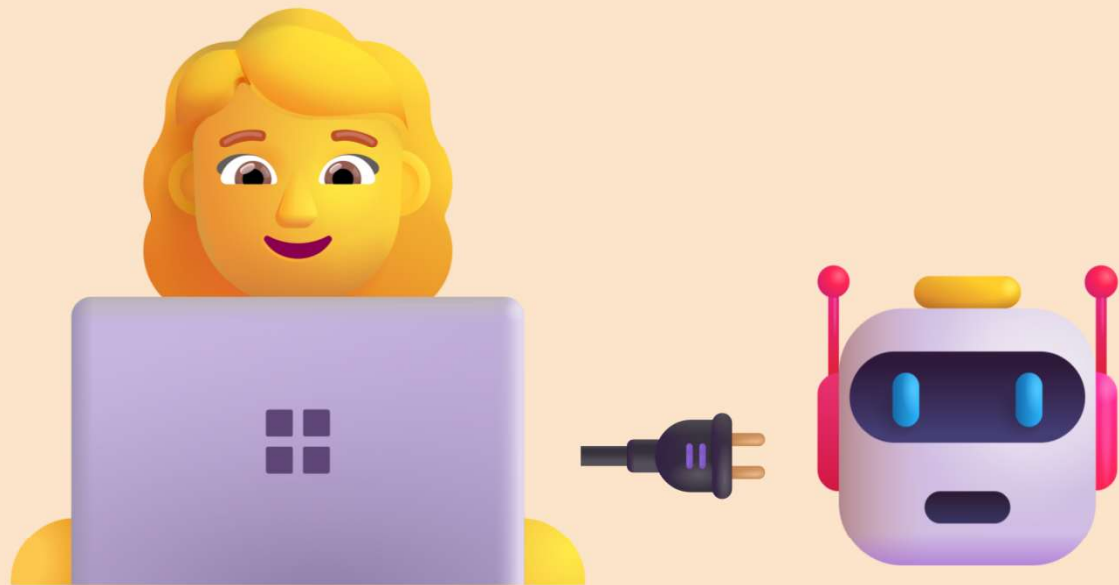


Stellt euch MCP wie einen zusätzlichen oder trainierten Muskel vor- ihr könnt zwar vllt. gekonnter bewegen oder schwerer heben aber davon weiß der Muskel noch lange nicht, was er da hebt geschweige denn, was er tut.

LLM-Fehler können durch MCP reale Handlungen auslösen.

Workshop

MCP – Model Context Protocol



Entscheidet euch bitte für euren Track:



Beginners-Track

...vorbereitet hier:

- Obsidian mit MCP – deine Notes mit AI-Superpower
- Blender mit MCP – 3D-Design in natürlicher Sprache
- Strudel.cc – schon mal Musik geskriptet?

Dein Startpunkt:

- Laptops aufklappen > Passwort = Username
- Starte in Obsidian
- Langflow – UI-geführte Agent/MCP Interaktion



Weitere Infos in Obsidian oder umfangreich im Repo auf GitHub (*beginners_track* Ordner)



Advanced-Track

...vorbereitet hier:

- Progressive Disclosure – Tool laden, nach Bedarf
- FastMCP – baue deinen eigenen MCP Server in Python
- Programmatic Tool Calling (PTC) – anderer Ansatz, gute Ergänzung?

Dein Startpunkt:

- Laptops aufklappen > Passwort = Username
- VSCode
- Gehe den *advanced_track* durch



Weitere Infos im Repo auf GitHub (*advanced_track* Ordner)

Legen wir los!



Lets get practical, practical...